

Projet Algorithmie 3

La distance de Jaccard

De - backer Cuvelier Sébastien Llobregat Nicolas

Version du 08 Avril 2025

Table des matières

1	Organisation du travail	3
1.1	Outils utilisés	3
1.2	Répartition du travail	3
2	Structure du projet	3
2.1	Les modules	3
2.2	Gestion des options	6
2.3	Gestion de l'entrée standart	6
3	Choix d'implémentation	6
3.1	Stockage des mots avec holdall	6
3.1.1	Implémentation par LDSC	6
3.1.2	Implémentation par tableaux dynamiques	6
3.2	Module op	6
4	Calcul de la distance de Jaccard	6
4.1	Module jdis	6
5	Les difficultés rencontrées	6
5.1	Tentative d'utilisation de getopt	6
5.2	La fonction fscanf	6
6	Performances	6
6.1	Complexité en espace	6
6.2	Complexité en temps	6
7	Améliorations possibles	6

1 Organisation du travail

1.1 Outils utilisés

Git & Github: Pour la gestion du projet et des différentes versions de celui-ci nous avons fait le choix d'utiliser un logiciel de versionning: **GIT**. Ainsi nous avons pu travailler en simultané sur les différentes composantes du projet. La methodology était la suivante: Créations des tâches via les issues **Github**. Puis création d'une branche spécifique pour chaque tâche à réaliser. Une fois le travail réaliser nous ouvrons une **pull request** pour que notre binôme puisse analyser le travail de l'autre avant de mutualiser sur la branche principale.

Tests unitaires: Nous avons pour ambition d'utiliser les tests unitaires pour s'assurer que chaque fonction réalisait bien ce qu'elle devait, pour se faire nous avons commencer à utiliser une bibliothèque qui permettait assez simplement de les réalilser: **greatest**. Cependant il s'est avéré très chronophage et nous avons pris du retard sur ces tests ce qui leur a fait perdre toute utilité. Nous avons donc choisi de ne pas les inclure dans notre version finale du projet.

1.2 Répartition du travail

Mesure de la charge de travail: Au début du projet nous n'avions pas réellement conscience de la charge de travail à effectuer. Il aurait peut-être été intéressant de lors de la première séance de réaliser un planning avec un tableau de répartition des tâches pour définir une charge de travail équitable. Cependant le projet n'étant pas de grande envergure et étant en effectif réduis à deux personnes nous avons pu communiquer efficacement pour communiquer sur les tâches que nous étions entrain de réaliser.

2 Structure du projet

Origine Lors de la conception de notre binôme nous avons décider de faire une lecture commune du sujet afin de savoir si nous avions la même vision. Ainsi nous avons organiser la structure du projet à l'origine puis nous nous sommes mis d'accord sur les modules à utiliser. Entre les deux implémentations suggérées soit l'utilisation de **tables de hachages** ou bien **d'abres binaires de recherches équilibrés**, c'est la deuxième qui nous a paru la plus adéquates. En effet il est plus facile de représenter les arbres et nous avons beaucoup plus travailler sur cette notion en cours.

2.1 Les modules

Les modules utilisés sont:

- Le module **avl**
- Le module **holdall**
- Le module **jdis**
- Le module **op**

Le module holdall Nous avons eu recours au module **holdall** pour le stockage des mots, afin de séparer le traitement des mots de la gestion en mémoire. Nous aurions pu choisir d'utiliser la fonction **bst_dft_infix_apply_context** du module **avl** et par exemple libérer les ressources en même temps que le dernier parcours de chaque arbre. Nous avons également décidé de tester cette solution en parallèle et nous en faisons référence dans la section 6 liée aux performances de l'implémentation proposée.

Le module jdis: Nous avons décidé de créer un module spécifique pour gérer la liaison entre le module **avl** et notre utilisation, il consiste principalement à convertir un fichier ou l'entrée standard (voir 2.3) en un **avl**. En le calcul du cardinal de l'intersection des ensembles de mots stockés dans les arbres. Et bien évidemment au calcul de la distance de Jaccard entre chaque couple deux à deux distincts des ensembles représentés par les arbres.

2.2 Gestion des options

2.3 Gestion de l'entrée standart

3 Choix d'implémentation

3.1 Stockage des mots avec holdall

3.1.1 Implémentation par LDSC

3.1.2 Implémentation par tableaux dynamiques

3.2 Module op

4 Calcul de la distance de Jaccard

4.1 Module jdis

5 Les difficultés rencontrées

5.1 Tentative d'utilisation de getopt

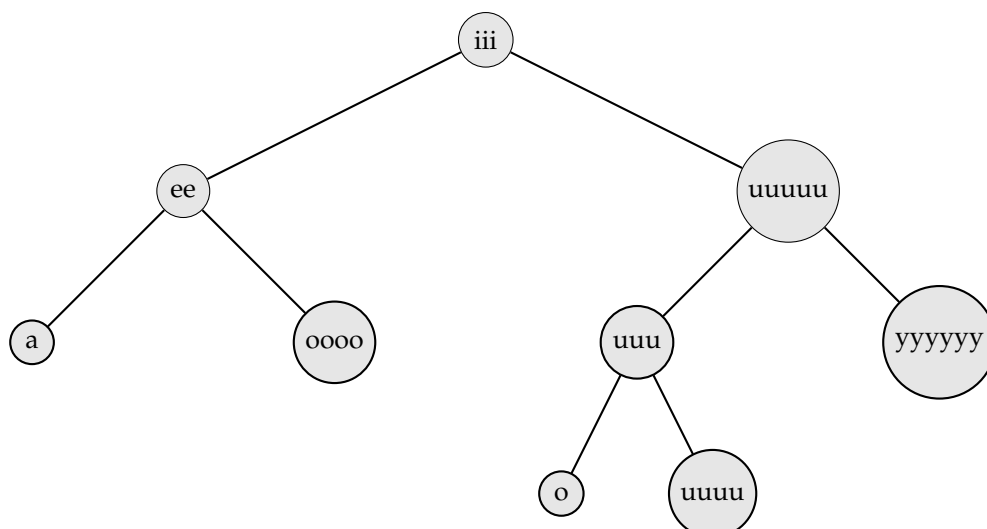
5.2 La fonction fscanf

6 Performances

6.1 Complexité en espace

6.2 Complexité en temps

7 Améliorations possibles



```
1 #include <stdio.h>
2
3 // Voici la fonction main.
4 int main() {
5     printf("Hello, _World!\n");
6     return 0;
7 }
```